



INTERNET SPEED TESTER



DOWNLOAD



UPLOAD



PING



JITTER



BY

DAKSH SAHU

PROJECT TYPE



PYTHON PROJECT



PYTHON



TKINTER



MATPLOTLIB



SPEEDTEST

INTERNET SPEED TESTER USING PYTHON

TABLE OF CONTENTS

SL. NO.	TOPIC	PG. NO.
1	Introduction	3
2	Objectives	3
3	Requirements Specification	4
4	Methodology	5
5	Implementation	6
6	Experimental Results	14
7	Applications	17
8	Future Scope of work	17
9	Conclusion	18
10	References	18

Introduction

Internet speed plays a crucial role in ensuring smooth browsing, online communication, and media streaming. With increasing dependence on cloud services and remote applications, it has become important for users to understand the performance of their network connection. An Internet speed test application serves this purpose by measuring key parameters such as download speed, upload speed, ping, and jitter, which together determine the overall quality and stability of the connection.

This project involves developing a feature-rich **Internet Speed Test Application using Python on a Windows PC**, built with the Tkinter GUI framework. The application integrates the speedtest module to measure real-time network speed and uses additional libraries such as psutil, requests, and socket to fetch system-related information, including IP addresses, MAC address, ISP details, and network type (Wi-Fi or Ethernet). A visually enhanced interface with dynamic speedometer-style gauges and live graphs is implemented using **Matplotlib** and **Pillow**, making the test results more interactive and user-friendly.

The system not only performs speed calculations but also displays network details such as client location, server selection, IPv4 and IPv6 addresses, and connection type. It includes additional features like jitter measurement, signal strength evaluation, data saving capability, graphical fluctuation tracking, and animated needles for download and upload speeds. This report explains the design, working principle, implementation, and output of the Internet speed test application.

Objectives

1. To develop a Python-based application that measures real-time **download speed, upload speed, ping, and jitter**.
2. To display detailed network information such as **ISP, IP addresses, MAC address, and connection type** (Wi-Fi/Ethernet).
3. To create a user-friendly GUI using **Tkinter** for running tests and viewing results.
4. To visualize speed results using **animated speedometer gauges and live fluctuation graphs**.
5. To allow users to **save test results** for future reference.
6. To detect network parameters automatically using system libraries such as psutil, socket, and requests.

System Requirements

Hardware Requirements

- PC or Laptop
- Minimum 4 GB RAM
- Stable Internet connection

Software Requirements

- **Operating System:** Windows 8 / 10 / 11, Mac, Linux, Ubuntu
- **Python Version:** Python 3.x
- **Code Editor:** Python IDLE or any other suitable IDE
- **Python Libraries Used:**
 - tkinter (built-in)
 - speedtest-cli
 - requests
 - psutil
 - Pillow
 - matplotlib
 - numpy
- **Image files required:**
 - back.png (background image)
 - speedometer_bg.jpg (gauge background)

Installation Commands:

Run the following commands in Command Prompt if on Windows:

```
pip install speedtest-cli
pip install requests
pip install psutil
pip install pillow
pip install matplotlib
```

Methodology

1. GUI Design:

The interface is built using Tkinter with a modern background, labels, buttons, and separate sections for network details and graph output.

2. Network Information Retrieval:

- requests is used to get public IP and ISP details.
- socket and psutil are used to obtain IPv4, IPv6, MAC address, and connection type.
- Time of test is automatically recorded.

3. Speed Measurement:

- The speedtest library selects the **best server** based on latency.
- Measures **ping**, **download**, and **upload** speeds.
- Jitter is calculated by measuring variations across multiple ping readings.

4. Speed Visualization:

- Matplotlib creates two speedometer-style gauges for download and upload.
- A third graph shows real-time speed fluctuations (history plot).
- Animated needles move gradually to the measured values.

5. Threading:

The speed test runs in a separate thread to prevent GUI freezing and ensure smooth animation.

6. Result Saving:

Users can save the results into a text file with all details including client info, speeds, signal strength, and time stamp.

7. Error Handling & Reset:

The application resets values to default when needed and handles network or library errors gracefully.

Implementation

Python Code:

```
import tkinter as tk
from tkinter import messagebox
import speedtest
import threading
import time
import math
import socket
import requests
from datetime import datetime
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from matplotlib.gridspec import GridSpec
import psutil
import numpy as np
from PIL import Image, ImageTk
import subprocess

class SpeedTestApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Internet Speed Test")
        self.root.geometry("950x650")

        # ==== BACKGROUND ====
        self.bg_original = Image.open("back.png")
        self.bg_image = ImageTk.PhotoImage(self.bg_original)
        self.bg_label = tk.Label(self.root, image=self.bg_image)
        self.bg_label.place(x=0, y=0, relwidth=1, relheight=1)
        self.root.bind("<Configure>", self.resize_background)

        # ==== COLORS ====
        self.CYAN = "#00F3FF"
        self.PURPLE = "#C400FF"
        BOX_BG = "#0b0013"
        BOX_BORDER = "#6f00ff"
        FONT_MAIN = ("Segoe UI", 10, "bold")

        # ==== TITLE ====
        title = tk.Label(
            root, text="INTERNET SPEED TEST",
            font=("Segoe UI", 16, "bold"),
            fg=self.CYAN, bg="#000000"
        ) title.pack(pady=10)

        # ==== INFO BOX ====
        info_box = tk.Frame(
            root, bg=BOX_BG,
            highlightbackground=BOX_BORDER,
            highlightthickness=3,
            width=520, height=34 )
        info_box.pack(pady=10)
        info_box.pack_propagate(True)

        # ==== 2-label system (Option B) ====
        def neon_label(title):
            frame = tk.Frame(info_box, bg=BOX_BG)
```

```

        lbl_title = tk.Label(
            frame, text=title,
            font=FONT_MAIN, fg=self.PURPLE, bg=BOX_BG )
        lbl_title.pack(side="left")
        lbl_value = tk.Label(
            frame, text="--",
            font=("Segoe UI", 10, "normal"), # normal font
            fg=self.CYAN, bg=BOX_BG)
        lbl_value.pack(side="left", padx=5)
        frame.pack(anchor="center")
        return lbl_title, lbl_value

# ==== LABELS ====
self.client_title, self.client_value = neon_label("Client:")
self.server_title, self.server_value = neon_label("Server(s):")
self.connection_title, self.connection_value = neon_label("Connection Type:")
self.local_ip_title, self.local_ip_value = neon_label("IPv4 Address:")
self.ipv6_title, self.ipv6_value = neon_label("IPv6 Address:")
self.mac_title, self.mac_value = neon_label("MAC Address:")
self.test_time_title, self.test_time_value = neon_label("Test Time:")
self.ping_title, self.ping_value = neon_label("Ping:")
self.jitter_title, self.jitter_value = neon_label("Jitter:")
self.download_title, self.download_value = neon_label("Download:")
self.upload_title, self.upload_value = neon_label("Upload:")
self.signal_title, self.signal_value = neon_label("Signal Strength:")

# ==== BUTTONS ====
btn_frame = tk.Frame(root, bg="#000000")
btn_frame.pack(pady=10)

def neon_button(text, cmd, color):
    btn = tk.Button(
        btn_frame, text=text, command=cmd,
        bg=color, fg="black",
        font=("Segoe UI", 10, "bold"),
        relief="flat", padx=15, pady=5)
    btn.bind("<Enter>", lambda e: btn.config(bg="white"))
    btn.bind("<Leave>", lambda e: btn.config(bg=color))
    return btn

self.start_btn = neon_button("START TEST", self.start_test, self.CYAN)
self.start_btn.pack(side=tk.LEFT, padx=10)
self.reset_btn = neon_button("RESET", self.reset_values, self.PURPLE)
self.reset_btn.pack(side=tk.LEFT, padx=10)
self.save_btn = neon_button("SAVE INFO", self.save_info, "#00FF90")
self.save_btn.pack(side=tk.LEFT, padx=10)

# ==== GRAPH AREA ====
self.fig = plt.Figure(figsize=(7,5), dpi=80)
gs = GridSpec(2, 2, figure=self.fig, height_ratios=[0.8,0.7], hspace=0.4, wspace=0.2)
self.ax_download = self.fig.add_subplot(gs[0,0])
self.ax_upload = self.fig.add_subplot(gs[0,1])
self.ax_history = self.fig.add_subplot(gs[1,:])
for ax in [self.ax_download, self.ax_upload]:
    ax.set_xlim(-1,1)
    ax.set_ylim(0,1)
    ax.axis('off')
    ax.margins(0)

self.ax_download.set_title("Download", color='cyan', fontsize=11)
self.ax_upload.set_title("Upload", color='magenta', fontsize=11)

```

```

img_download = mpimg.imread("speedometer_bg.jpg")
img_upload = mpimg.imread("speedometer_bg.jpg")
self.ax_download.imshow(img_download, extent=[-1,1,0,1], zorder=0)
self.ax_upload.imshow(img_upload, extent=[-1,1,0,1], zorder=0)

x0, y0 = self.draw_needle(0)
self.download_line, = self.ax_download.plot(x0, y0, lw=2.5, color='cyan', zorder=1)
self.upload_line, = self.ax_upload.plot(x0, y0, lw=2.5, color='magenta', zorder=1)
self.ax_history.set_facecolor('white')
self.ax_history.set_title("Speed Fluctuations", color='black', fontsize=11)
self.ax_history.set_xlabel("Time Steps", color='black', fontsize=9)
self.ax_history.set_ylabel("Mbps", color='black', fontsize=9)
self.ax_history.set_xlim(0,100)
self.ax_history.set_ylim(0,100)

self.download_history = [0]
self.upload_history = [0]
self.line_download_hist, = self.ax_history.plot(self.download_history, color='cyan',
label='Download')
self.line_upload_hist, = self.ax_history.plot(self.upload_history, color='magenta',
label='Upload')
legend = self.ax_history.legend(fontsize=8)
for text in legend.get_texts():
    text.set_color("black")
self.fig.subplots_adjust(left=0.08, right=0.95, top=0.92, bottom=0.12)
self.canvas = FigureCanvasTkAgg(self.fig, master=root)
self.canvas.get_tk_widget().pack(pady=8)
self.canvas.draw()
self.running = False

def resize_background(self, event):
    if event.width < 200 or event.height < 200: return
    resized = self.bg_original.resize((event.width, event.height), Image.LANCZOS)
    self.bg_image = ImageTk.PhotoImage(resized)
    self.bg_label.config(image=self.bg_image)

def draw_needle(self, value, max_value=100, radius=0.9):
    angle = math.pi * (1 - min(value, max_value) / max_value)
    x = [0, radius * math.cos(angle)]
    y = [0, radius * math.sin(angle)]
    return x, y

def detect_connection_type(self):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()

        addrs = psutil.net_if_addrs()
        for iface_name, iface_addrs in addrs.items():
            for addr in iface_addrs:
                if addr.family == socket.AF_INET and addr.address == local_ip:
                    name_lower = iface_name.lower()
                    if "wi-fi" in name_lower or "wireless" in name_lower or "wlan" in
name_lower:
                        return "Wi-Fi"
                    else: return "Ethernet"
            return "Unknown"
    except: return "Unknown"

```



```

def get_ipv6_address(self):
    try:
        addrs = psutil.net_if_addrs()
        for iface in addrs:
            for addr in addrs[iface]:
                if addr.family == socket.AF_INET6:
                    ipv6 = addr.address.split('%')[0]
                    if ipv6 != "::1":
                        return ipv6
    except: pass
    try:
        ipv6 = requests.get("https://api64.ipify.org", timeout=3).text
        if ipv6:
            return ipv6
    except:
        pass
    return "Not Available"

def get_mac_address(self):
    try:
        addrs = psutil.net_if_addrs()
    except:
        return "Not Available"

    for iface, addr_list in addrs.items():
        for addr in addr_list:
            if addr.family == psutil.AF_LINK:
                mac = addr.address
                if mac and mac != "00:00:00:00:00:00":
                    return mac.upper()
    return "Not Available"

def start_test(self):
    if not self.running:
        self.running = True
        threading.Thread(target=self.run_speedtest, daemon=True).start()

def run_speedtest(self):
    # Client info
    try:
        response = requests.get("https://ipinfo.io/json", timeout=5).json()
        public_ip = response.get('ip', 'Unknown')
        isp = response.get('org', 'Unknown')
        city = response.get('city', 'Unknown')
        country = response.get('country', 'Unknown')
    except: public_ip = isp = city = country = "Unknown"
    self.root.after(0, lambda: self.client_value.config(
        text=f"{city}, {country} | {public_ip} {isp}" ))
    # Connection type
    conn = self.detect_connection_type()
    self.root.after(0, lambda: self.connection_value.config(text=conn))

    # IPv4
    try:
        ipv4 = socket.gethostbyname(socket.gethostname())
    except:
        ipv4 = "Unknown"
    self.root.after(0, lambda: self.local_ip_value.config(text=ipv4))

    # IPv6

```

```

    ipv6 = self.get_ipv6_address()
    self.root.after(0, lambda: self.ipv6_value.config(text=ipv6))

    # MAC
    mac = self.get_mac_address()
    self.root.after(0, lambda: self.mac_value.config(text=mac))

    # Time
    test_time = datetime.now().strftime("%d-%b-%Y %H:%M:%S")
    self.root.after(0, lambda: self.test_time_value.config(text=test_time))

    # Speedtest
    st = speedtest.Speedtest()
    server = st.get_best_server()

    server_text = f"{server['name']}, {server['country']} | {server['sponsor']}"
    self.root.after(0, lambda: self.server_value.config(text=server_text))

    # ping
    self.root.after(0, lambda: self.ping_value.config(text=f"{server['latency']} ms"))

    # jitter
    pings = []
    for _ in range(5):
        st.get_best_server()
        pings.append(st.results.ping)
        time.sleep(0.1)

    jitter = sum(abs(pings[i] - pings[i+1]) for i in range(len(pings)-1)) / (len(pings)-1)
    self.root.after(0, lambda: self.jitter_value.config(text=f"{jitter:.2f} ms"))

    # download
    download = st.download() / 1_000_000
    self.animate_needle(download, self.download_line, self.download_value, "Download")

    # upload
    upload = st.upload() / 1_000_000
    self.animate_needle(upload, self.upload_line, self.upload_value, "Upload")

    # signal
    if download < 1:
        sig = "Poor"
    elif download < 5:
        sig = "Average"
    elif download < 20:
        sig = "Good"
    elif download < 30:
        sig = "Very Good"
    else:
        sig = "Excellent"

    self.root.after(0, lambda: self.signal_value.config(text=sig))
    self.running = False

def animate_needle(self, target_speed, line, label, text):
    steps = 50
    for i in range(steps):
        spd = target_speed * (i + 1) / steps
        x, y = self.draw_needle(spd)
        line.set_data(x, y)
        self.root.after(0, lambda s=spd: label.config(text=f"{s:.2f} Mbps"))

```

```

        if text == "Download":
            self.download_history.append(spd)
            self.upload_history.append(self.upload_history[-1])
        else:
            self.upload_history.append(spd)
            self.download_history.append(self.download_history[-1])

        self.download_history = self.download_history[-100:]
        self.upload_history = self.upload_history[-100:]

        self.line_download_hist.set_data(range(len(self.download_history)),
self.download_history)
        self.line_upload_hist.set_data(range(len(self.upload_history)), self.upload_history)

        self.ax_history.set_xlim(0, 100)
        self.ax_history.set_ylim(0, max(max(self.download_history), max(self.upload_history),
50))

        self.canvas.draw()
        time.sleep(0.05)

    def reset_values(self):
        fields = [
            self.client_value, self.server_value, self.connection_value,
            self.local_ip_value, self.ipv6_value, self.mac_value,
            self.test_time_value, self.ping_value, self.jitter_value,
            self.download_value, self.upload_value, self.signal_value]
        for f in fields:
            f.config(text="--")
        x0, y0 = self.draw_needle(0)
        self.download_line.set_data(x0, y0)
        self.upload_line.set_data(x0, y0)
        self.canvas.draw()

    def save_info(self):
        try:
            test_time = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
            filename = f"InternetSpeedTest_{test_time}.txt"

            with open(filename, "w") as f:
                f.write(f"Client: {self.client_value.cget('text')}\n")
                f.write(f"Server(s): {self.server_value.cget('text')}\n")
                f.write(f"Connection Type: {self.connection_value.cget('text')}\n")
                f.write(f"IPv4 Address: {self.local_ip_value.cget('text')}\n")
                f.write(f"IPv6 Address: {self.ipv6_value.cget('text')}\n")
                f.write(f"MAC Address: {self.mac_value.cget('text')}\n")
                f.write(f"Test Time: {self.test_time_value.cget('text')}\n")
                f.write(f"Ping: {self.ping_value.cget('text')}\n")
                f.write(f"Jitter: {self.jitter_value.cget('text')}\n")
                f.write(f"Download: {self.download_value.cget('text')}\n")
                f.write(f"Upload: {self.upload_value.cget('text')}\n")
                f.write(f"Signal Strength: {self.signal_value.cget('text')}\n")

            messagebox.showinfo("Saved", f"Test info saved to {filename}")
        except Exception as e:
            messagebox.showerror("Error", str(e))

# Run App
root = tk.Tk()
app = SpeedTestApp(root)
root.mainloop()

```

Overview of Code:

The Internet Speed Test Application is implemented in Python using a combination of GUI, networking, and visualization libraries. The implementation consists of four major components: user interface design, network information extraction, speed testing logic, and graphical visualization.

1. Graphical User Interface (GUI)

The GUI is created using **Tkinter**.

Key elements include:

- A background image loaded through **Pillow (PIL)**
- Labels for displaying client and network information
- Buttons for **Start Test**, **Reset**, and **Save Info**
- A central information box for all measured parameters
- Embedded Matplotlib graphs for gauges and speed history

The layout is responsive and updates dynamically based on window size.

2. Network Information Retrieval

Several libraries are used to collect system and network data:

- **requests** → fetches public IP, city, country, and ISP via ipinfo.io
- **socket** → gets local IPv4 address
- **psutil** → retrieves IPv6, MAC address, and identifies connection type (Wi-Fi/Ethernet)
- **datetime** → records the test timestamp

The application updates all data labels asynchronously using Tkinter's `after()` method.

3. Speed Testing Logic

The internet speed test is performed using the **speedtest** module.

Steps:

1. Select the **best server** automatically using `get_best_server()`.
2. Measure **latency (ping)** from server details.
3. Calculate **jitter** by taking multiple ping values and computing variation.
4. Measure **download speed** and **upload speed** in Mbps.
5. Assign **signal strength** (Poor / Average / Good / Very Good / Excellent) based on download value.

To prevent the GUI from freezing, the entire test runs inside a **background thread**

4. Speedometer Needle Animation

The animated speedometer effect is created using Matplotlib:

- A static speedometer background image (`speedometer_bg.jpg`) is displayed.
- A needle is drawn using a line whose angle changes based on speed.
- The function `draw_needle()` converts speed values into rotation angles.
- The `animate_needle()` method gradually moves the needle from 0 to the target speed in 50 steps.
- Both **download** and **upload** gauges animate independently.

This creates a smooth, real-time visual effect.

5. Speed Fluctuation Graph

A live graph of speed history is plotted using Matplotlib's Axes:

- Two lists track download and upload data values.
- Only the latest 100 points are kept for clarity.
- The graph updates on every needle movement.
- The Y-axis adjusts dynamically based on the highest recorded speed.

This helps visualize speed fluctuations clearly during the test.

6. Data Reset and Saving

- **Reset Feature:**

Resets all labels and needles to default values.

- **Save Info Feature:**

The application generates a .txt file containing:

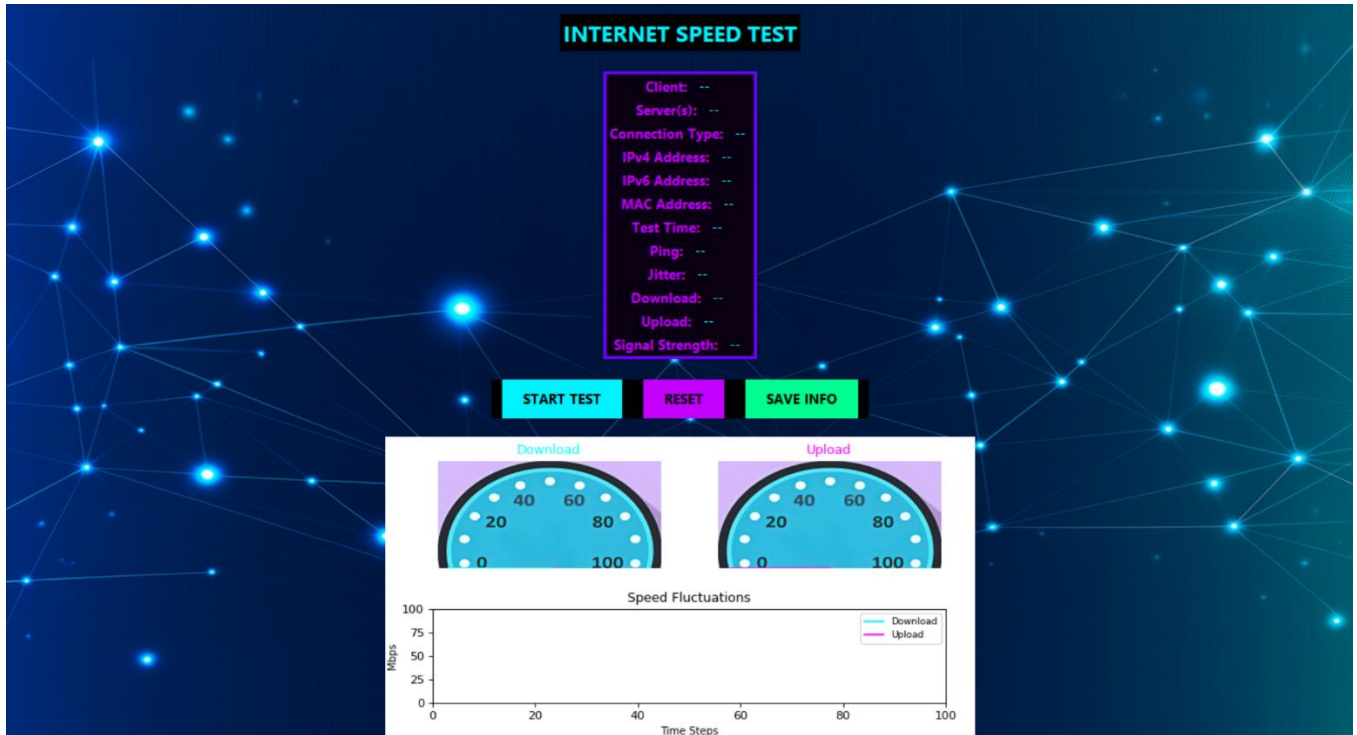
- Client details
- Server used
- IP addresses (IPv4 & IPv6)
- MAC address
- Ping, jitter, download, upload
- Signal strength
- Timestamp

The file name is generated using the current date and time.

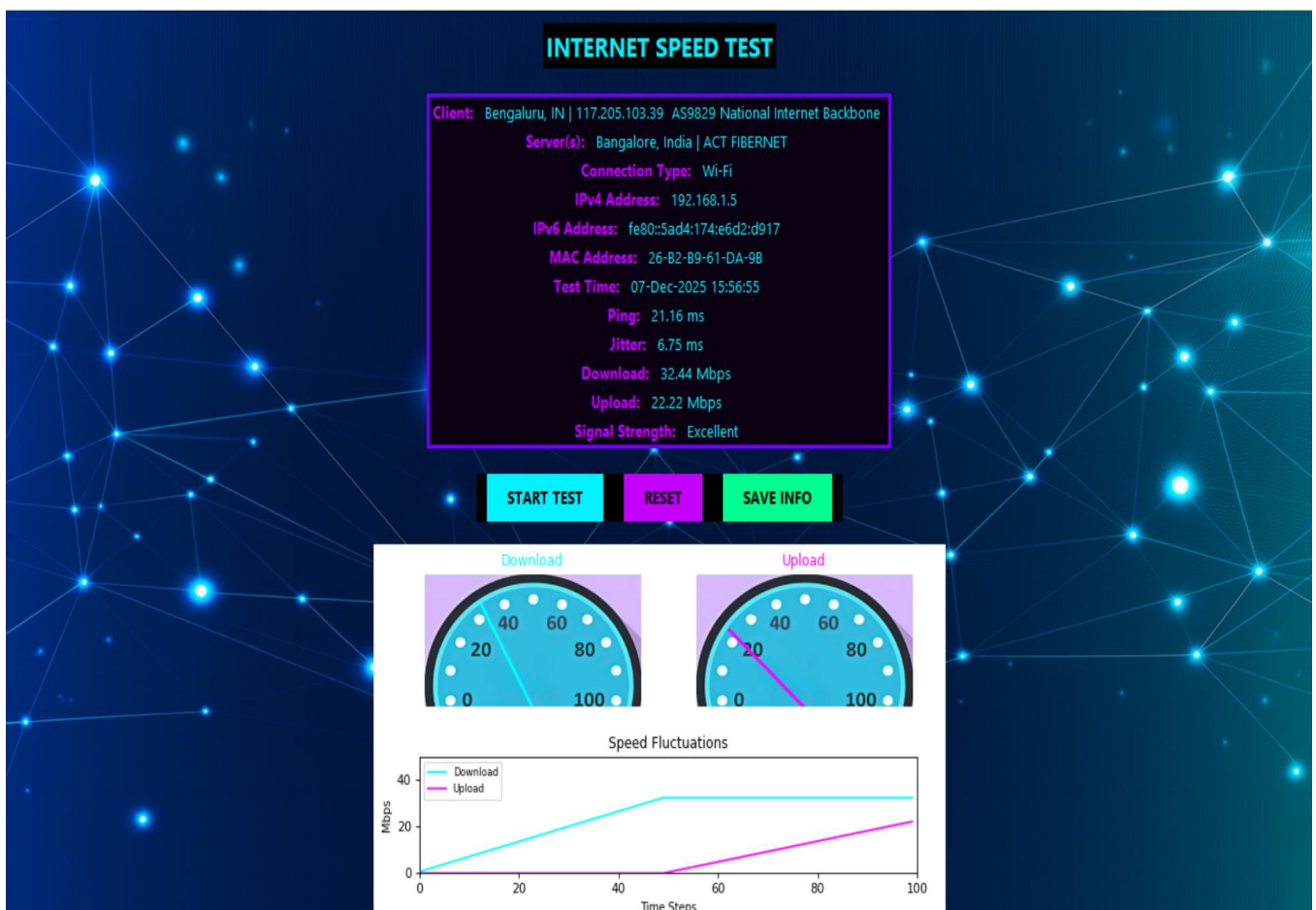
7. Thread and Error Handling

- Threading ensures the GUI remains responsive.
- Errors during requests, speed test, or file saving are handled using Tkinter messagebox.
- All UI updates are done through root.after() to avoid thread-UI conflicts.

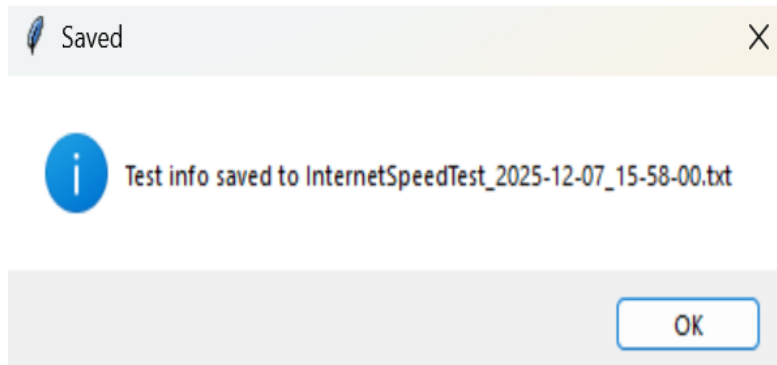
Experimental Results



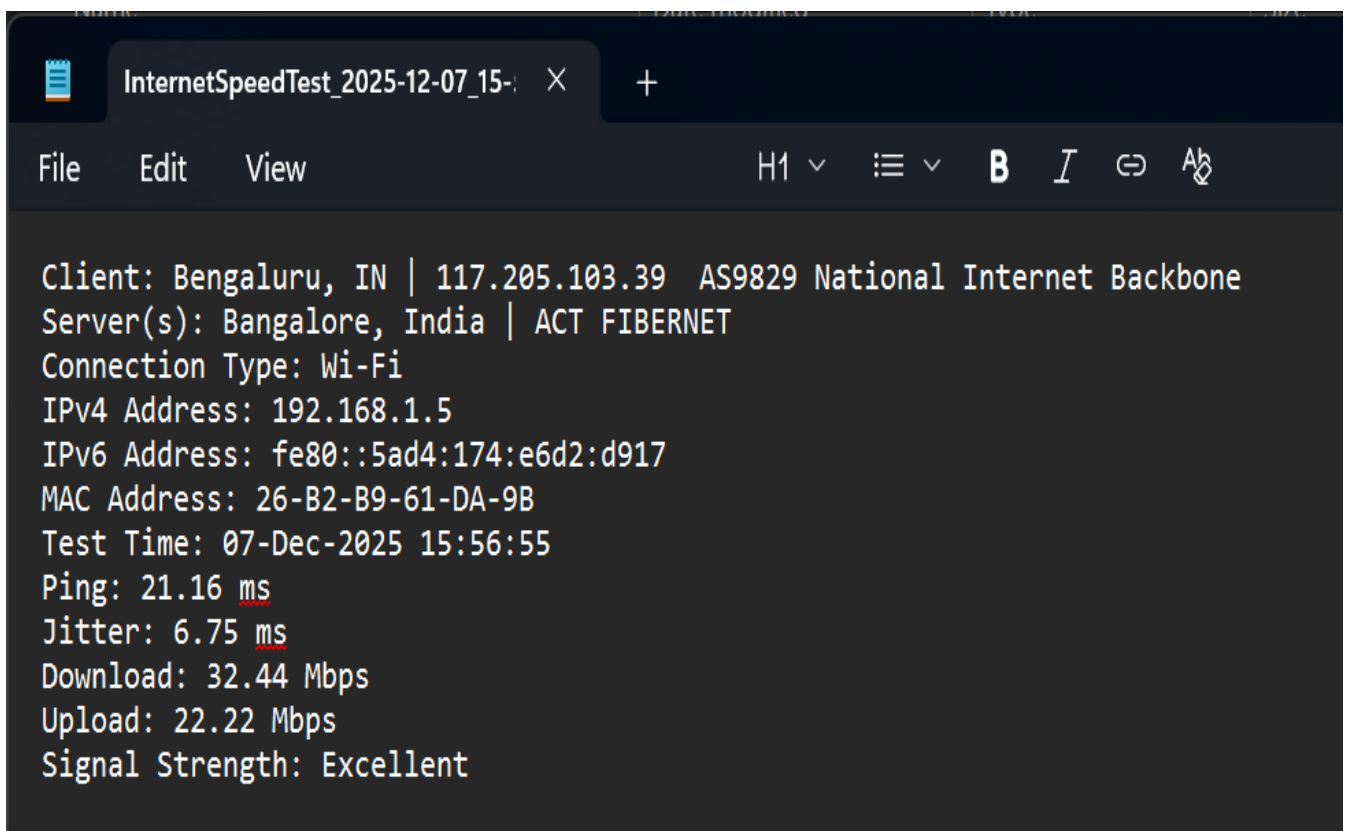
Opening the application



After running a speed test



Saving test results



Opening the .txt file for info

Overview of Output:

The output of the Internet Speed Test application is divided into different sections that show information about the network, the test results, and visual representations of speed performance.

1. Network Information Section

This part displays overall details about the user's internet connection. It includes information such as the *client location*, the *internet service provider (ISP)*, the *server selected for the test*, and *technical details like IP addresses, MAC address, and connection type*. It helps the user understand which network and server were used during the test.

2. Test Result Section

This section shows the main metrics measured during the speed test:

- **Ping** – how quickly the device communicates with the server (latency).
- **Jitter** – stability of the connection.
- **Download Speed** – how fast data is received from the internet.
- **Upload Speed** – how fast data is sent to the internet.
- **Signal Strength** – quality of the Wi-Fi or network connection.

These values help users understand the performance of their network.

3. Control Buttons

Buttons like *Start Test*, *Reset*, and *Save Info* allow the user to run a new test, clear the current results, or save the information for future reference.

4. Speed Gauges

Two graphical meters (speedometers) are used to visually represent download and upload speeds. They give an easy-to-understand view of how fast the internet is performing.

5. Speed Fluctuation Graph

A line graph is displayed to show how the download and upload speeds changed during the test. This helps users see if their internet speed is stable or fluctuates over time.

Applications

1. **Home and Personal Use:**

Helps users quickly check the performance of their home Internet connection, including speed, latency, and reliability.

2. **Network Troubleshooting:**

Useful for diagnosing issues such as slow connections, high ping, unstable networks, or ISP-related problems.

3. **ISP Performance Verification:**

Allows users to verify whether their Internet Service Provider delivers the promised bandwidth.

4. **Educational Use:**

Demonstrates practical implementation of networking concepts, GUI development, and Python programming in academic projects.

5. **IT Support and Maintenance:**

Can be used by technical support teams to measure network health and generate test logs.

6. **Performance Monitoring:**

The live speed fluctuation graph helps monitor network stability over time.

Future Scope of the Work

- The application can be enhanced to store all speed test results in a file or database for long-term tracking.
- A notification system can be added to alert users when their internet speed drops below a certain limit.
- The GUI can be improved with additional themes, animations, and a more modern interface.
- Support for scheduled automatic speed tests can be added to check internet performance at regular intervals.
- The application can be extended to generate reports and graphs.
- A mobile or web version of the speed test tool can be developed for wider accessibility.

Conclusion

The Internet Speed Test Application developed using Python provides an efficient, user-friendly, and visually interactive way to measure real-time network performance. Through the integration of libraries such as speedtest, tkinter, psutil, requests, and matplotlib, the application successfully captures essential parameters like download speed, upload speed, ping, jitter, IP addresses, and connection type.

The inclusion of animated speedometers, dynamic graphs, and automatic data saving enhances both usability and functionality.

Overall, the project demonstrates how Python can be used to build a complete desktop application that combines GUI design, system networking, and data visualization. It serves as a practical tool for users to understand and evaluate their Internet connection quality effectively.

References

1. **Python Documentation:**
<https://docs.python.org/>
2. **Speedtest-cli Library Documentation:**
<https://github.com/sivel/speedtest-cli>
3. **Tkinter GUI Programming Guide:**
<https://docs.python.org/3/library/tkinter.html>
4. **Matplotlib Official Documentation:**
<https://matplotlib.org/stable/>
5. **Psutil Documentation:**
<https://psutil.readthedocs.io/>
6. **Requests Library Documentation:**
<https://requests.readthedocs.io/>
7. Stack Overflow Community Discussions
(for troubleshooting and common implementation solutions)